

GET ADAPTER INFORMATION

The `GetAdapterInformation` method can then be called once the adaptor's unique ID is available. This can be used in order to query a host of information such as the adaptor's name, version, state, media or signal among others. We can pass the `UniqueID` and the `bsKey` for the required information, which will then return the value in its output parameter. An example of calls to such a method is as follows: -

```
WiDi.GetAdapterInformation(sAdapterId, keyName, out valueName);
```

START CONNECTION TO ADAPTOR

The final step for establishing a connection is to call this method by passing in the adaptor's unique ID, screen resolution of the Windows host to be used, the resolution information about the TV or the display screen and the screen mode to be used.

```
WiDi.StartConnectionToAdapter(sAdapterId, 0, 0, smConnectMode);
```

The zero value for resolution indicates that a default resolution is to be used. If the adaptor to which a connection is requested, has not already been paired, then the authentication UI will be shown, expecting the user to enter authentication information.

SET SCREEN MODE

This method can be called to change the display mode of the current screen to local only, extended mode, clone mode or TV-only.

SET CLIPPING RECTANGLE

If a limited area of the screen needs to be reflected on the display screen, then this method can be used to set the co-ordinates of the area of the screen that is to be transmitted. Input parameters are the X, Y coordinates and the width/height of the rectangle.

DISCONNECT ADAPTER

A call to this method is made once the connection to a particular adaptor needs to be cancelled. We may or may not specify the adaptor's Unique ID. If the unique ID is not specified (we can pass the null value as "" instead). The current connection is disconnected irrespective of whichever adaptor is connected.

```
WiDi.DisconnectAdapter(m_sConnectedAdapter);
```

SHUTDOWN

Finally, at the close of the application, the shutdown method is called once in order to close all mechanisms in order to make connections to WiDi Adaptors.

Setting up the IDE for the Sample WiDi Application in C# 2010

1. Create a new Windows Forms Application
2. Right-click on References in the Solution Explorer and choose "Add Reference..."
3. In the window that appears, switch to the COM tab
4. Select "Intel WiDi Extensions SDK 1.0 Type Library" in the list under the Component Name column and press OK.
5. Under References in the Solution Explorer, you should now see an item named "IntelWiDiLib". Once added, the interface can be explored further from the Object Browser. The DLL file will be placed in the project's output directory and needs to be packaged and registered when the application is installed.
6. Double-click on the background of your form to create an event handler for the Form Load event.
7. Type the following line of code:


```
IntelWiDiLib.WiDiExtensions
WiDi = new IntelWiDiLib.WiDiExtensions();
```
8. On the next line, type "WiDi." and a drop-down should appear which shows you all the methods of the class which are avail-

able. This method can be used to call all the functions that can be performed by this SDK. Now, the sample application can be developed further to provide WiDi functions to the application.

Setting up the IDE for a sample WiDi Application in Visual C++

1. Create a new MFC Application and follow through the wizard to set up the new project as you would like.
2. Add the following lines of code in the `InitInstance` method of the MFC application class (if they are not already present).


```
If (!AfxOleInit()) //Initialize COM
{
    return FALSE; //Unable to Initialize COM
}
If the AfxOleInit() function is not recognized, you will need to include<afxdisp.h> in your precompiled header file (stdafx.h)
```
3. In the menu, go to `Project > Add Existing Item...` and a dialog will appear for adding files to the project. Choose the `IntelWiDiExtensions.idl` file (included with SDK) to add it to the project.
4. Find the newly added `IntelWiDiExtensions.idl` file in Solution Explorer, under Source Files. Right-click it and choose `Compile`. The IDL file should compile with no errors.
5. Once compilation is complete, there will be two new files available: `IntelWiDiExtensions_i.h`



A typical Wi-Di setup where a laptop screen is being mirrored to a TV